

OpenVisTool: An Open Recipe for Synthesizing Instructive Visual Tool-Use Trajectories

Changhao Xiang^{1,2*†}, Shilin Zhang^{1,2*†}, Zheng Ma^{2‡}, Kanzhi Cheng¹, Ruize Ma¹, Yi Feng¹, Jianbing Zhang¹, Zhi Wang^{1§}, Zhen Wu^{1§}, Xinyu Dai¹, Lewei Lu²

¹Nanjing University ²SenseTime Research

{xiangch, shilinzhang}@smail.nju.edu.cn {zhiwang, wuz}@nju.edu.cn

Abstract

Visual tool use has emerged as a fundamental capability for multimodal agents to actively acquire evidence beyond a fixed image encoding. The prevailing recipe learns this capability from teacher-generated trajectories filtered for answer correctness, implicitly assuming that every successful demonstration provides effective supervision. We argue this assumption is flawed: a strong teacher often reaches the correct answer without needing its tool calls, and imitating such trajectories teaches a student that tool calls accompany correct answers, not that tool observations ground them. We present **OpenVisTool**, an open framework for constructing *instructive visual tool-use trajectories* that provide effective supervision for tool learning. The key insight is that a trajectory should be retained only if its answer is correct (outcome validity) and its tool observations causally contribute to that answer (causal utility). The framework operates in three stages: *difficulty screening* to select queries that are not reliably answerable without tools, *domain-specific trajectory synthesis* to elicit coherent tool-use trajectories, and *supervision verification* to jointly test both conditions. Rather than encouraging models to imitate tool calls, the resulting supervision teaches when and how visual evidence should be acquired. Using this framework, we construct **OpenVisTool-42K**, a dataset spanning five visual reasoning domains, together with **OpenVisTool-Bench**, a benchmark covering the same domains. Across four backbones (4B–27B), fine-tuning on OpenVisTool-42K consistently improves visual tool-use performance and yields gains on two out-of-distribution benchmarks; the larger models approach leading closed-source systems. The evidence suggests that effective visual tool use is learned from causally grounded supervision rather than tool-calling patterns. Our code is available at <https://github.com/Changhao-Xiang/OpenVisTool>.

1 Introduction

Large language models become more capable when equipped with external tools such as code interpreters (Xue et al. 2025; Feng et al. 2025) and search engines (Jin et al. 2025; Li et al. 2025b,c). For multimodal models, the bottleneck is visual: encoding an image into a fixed set of tokens inevitably discards the fine-grained evidence that many questions hinge on (Su et al. 2025c). Visual tools recover such evidence by feeding new observations into reasoning—cropping to reveal small

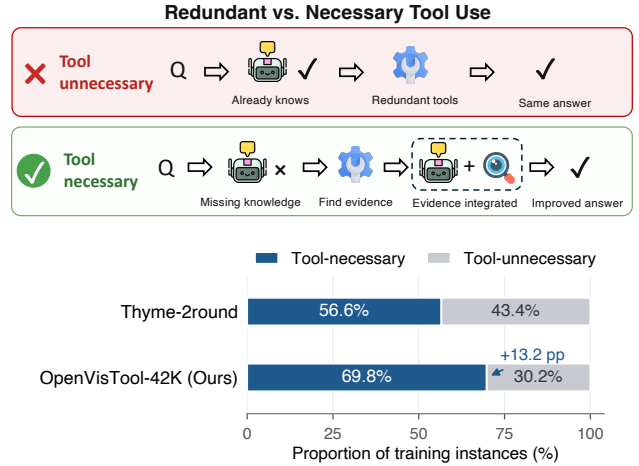


Figure 1: Correct tool-use trajectories are not always instructive. (Top) Illustration of tool-unnecessary and tool-necessary trajectories. (Bottom) Tool-necessity distributions for Thyme-2round and our OpenVisTool-42K, computed over 1,000 instances by re-querying the corresponding teacher model without tool access. Trajectories with correct no-tool answers are labeled *tool-unnecessary*.

text (Kaur et al. 2026), zooming in to resolve spatial details (Zheng et al. 2026), or rendering generated code against a reference image (Yang et al. 2026)—a paradigm known as *thinking with images* (OpenAI 2025). To instill this capability, recent systems couple cold-start supervised fine-tuning (SFT) on teacher-generated trajectories with reinforcement learning (RL) that refines the policy (Lai et al. 2026; Hou et al. 2025; Guo et al. 2025). Since RL alone rarely bootstraps stable tool invocation (Hong et al. 2026; Zhang et al. 2026), the quality of the cold-start data largely determines a model’s ultimate tool-use competence. However, not all trajectories contribute equally, and *what makes one genuinely valuable for learning* remains an open question.

Existing synthesis pipelines typically begin with outcome-based rejection sampling: a trace is retained if it is executable, well-formed, and yields a correct answer (Lai et al. 2026;

*These authors contributed equally.

†Work done during internships at SenseTime Research.

‡Project leader.

§Corresponding author.

Hong et al. 2026). This criterion, however, conflates outcome validity with the utility of the tool calls behind it. Some pipelines further employ model or human judges to verify that tool outputs are consistent with the reasoning and plausibly support the answer (Zhang et al. 2026, 2025). Yet such judgments still do not establish counterfactual utility. A strong teacher can often reach the right answer without tools (via parametric knowledge, reasoning shortcuts, or coarse image cues), yet still issues tool calls simply because the synthesis prompt instructs it to (Hou et al. 2025; Zhao et al. 2026). Such trajectories pass both outcome- and judge-based filtering while providing *spurious supervision*: the visual observations appear relevant but make no causal contribution to the answer. A student trained on such data learns correlation rather than causation: it learns that tool calls *accompany* correct answers, not that tool observations *ground* them. Answering the open question above thus requires shifting the object of evaluation from whether a trajectory ends correctly to whether its tool calls do real work: *would the answer still hold if the visual observations were taken away?*

Our answer is a two-part criterion: a trajectory constitutes beneficial supervision for visual tool-use learning only when it satisfies two conditions jointly. The first is *outcome validity*: the trajectory must reach a correct final answer, so that the model is not trained on erroneous reasoning chains. The second, which existing pipelines overlook, is *causal utility*: the trajectory’s tool trace must improve a fixed probe model’s answer reliability relative to a no-tool baseline, with the original image and query held constant. We estimate this utility by comparing the probe model’s success rate when conditioned on the recorded tool calls and observations against its success rate without them. A trajectory meeting both conditions teaches the model not merely that tools *can* be called, but *why* a call is worth making: it supplies visual evidence that the model’s intrinsic encoding cannot. This criterion is not merely conceptual but directly testable. We build our data synthesis pipeline around it.

Guided by this criterion, we design **OpenVisTool**, a three-stage trajectory synthesis pipeline that puts causal supervision selection into practice across five visual reasoning domains (chart, table, GUI grounding, visual search, and web-to-HTML) under a shared visual toolset. First, we apply *Difficulty Screening* to retain only questions that a capable model fails to answer reliably without tool access, so that the surviving problems genuinely demand external visual evidence. Second, during *Domain-Specific Trajectory Synthesis*, we use structured invocation strategies tailored to each domain to guide the teacher toward coherent traces rather than arbitrary tool calls. Finally, we perform *Supervision Verification* to ensure that each trajectory satisfies both conditions above: it is retained only if its final answer is correct (outcome validity) and its tool traces, when provided to a base model, improve its answer (causal utility). Together, the three stages impose progressively stricter requirements: a question must demand visual tools, a trace must invoke them coherently, and an answer must causally depend on what the tools return. In summary, our contributions are threefold:

- We introduce **OpenVisTool**, a three-stage framework for

constructing instructive visual tool-use trajectories by jointly enforcing outcome validity and causal utility.

- We construct **OpenVisTool-42K**, an open-source large-scale dataset spanning five representative visual reasoning domains under a shared visual toolset.
- We fine-tune visual tool agents on four backbones (4B–27B) using OpenVisTool-42K. The resulting models are competitive with leading closed-source models, generalize to out-of-distribution tasks, and outperform models trained with alternative data-filtering strategies.

2 Related Work

Visual Tool-Use Learning Recent work on training visual tool-use agents follows two paradigms. One directly optimizes tool-calling policies via reinforcement learning (Zheng et al. 2026; Wu et al. 2026), but RL alone often fails to bootstrap stable tool invocation without a well-initialized policy. The dominant recipe therefore adopts a two-stage approach: cold-start supervised fine-tuning (SFT) on teacher-generated trajectories to establish basic tool-use competence, followed by RL for further refinement (Su et al. 2025b,a; Hong et al. 2026; Lai et al. 2026; Guo et al. 2025; Hou et al. 2025). Under this paradigm, SFT quality directly determines whether RL can converge to capable policies (Hong et al. 2026; Zhang et al. 2026). Yet existing work largely treats the SFT corpus as a commodity—any answer-correct teacher trajectory is assumed to provide useful supervision. What constitutes genuinely beneficial supervision for visual tool-use learning has not been systematically studied.

Supervision Quality for Reasoning In text-based reasoning, the quality of training data has received sustained attention. Rejection sampling—retaining only solutions that reach the correct final answer—is a standard recipe for curating reasoning corpora (Zelikman et al. 2022; Yuan et al. 2023). Subsequent studies demonstrate that not all correct traces are equally instructive: process-level verification reveals that individual reasoning steps vary in correctness and informativeness (Lightman et al. 2024; Wang et al. 2024a), and difficulty-aware or diversity-aware selection further improves learning efficiency over naive outcome filtering (Yu et al. 2024). These findings establish a clear lesson: supervision quality matters as much as quantity. However, this lesson has yet to be transferred to the visual tool-use setting, where the notion of “quality” is further complicated by tool observations—a returned modality whose actual contribution to reasoning is neither guaranteed nor straightforward to assess.

Visual Evidence Acquisition The paradigm of “thinking with images,” introduced by OpenAI’s o3 (OpenAI 2025), reframes visual reasoning as an active evidence-acquisition process: rather than relying on a single static encoding, models iteratively crop, zoom, or render visual inputs and incorporate the resulting observations into their reasoning (Su et al. 2025c). Under this lens, visual tools acquire evidence that the model’s intrinsic visual encoding cannot provide. This perspective motivates our causal framing: a trajectory is instructive only when its tool observations causally contribute to reaching the correct answer. Trajectories in which tools are

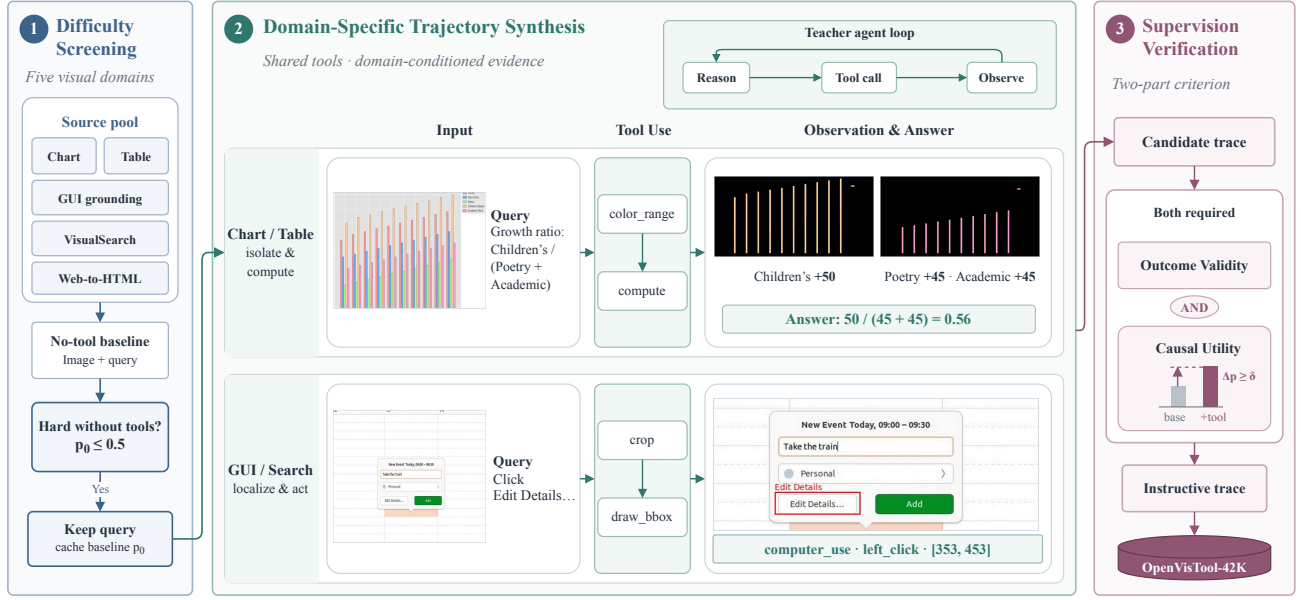


Figure 2: Overview of the three-stage OpenVisTool data pipeline. **Difficulty Screening** identifies queries that the probe model cannot reliably solve from the original image alone. **Domain-Specific Trajectory Synthesis** uses domain-specific tool-use patterns to guide a stronger teacher in acquiring evidence that addresses each domain’s visual bottleneck. **Supervision Verification** determines whether each trajectory is instructive: it must achieve a valid outcome, and its tool observations must causally improve the probe model’s answer reliability. Only trajectories satisfying both criteria are retained.

invoked but the answer could have been obtained without the returned observations teach *correlation*—tool calls co-occur with success—rather than the deeper skill of knowing *when and why* to seek external visual evidence.

3 OpenVisTool

3.1 What Makes a Trajectory Instructive?

Consider a visual reasoning problem with input image I , query q , and reference answer y^* . A candidate tool-use trajectory is represented as

$$\tau = ((r_t, c_t, o_t)_{t=1}^T, \hat{y}_\tau), \quad (1)$$

where T is the number of turns, r_t , c_t , and o_t denote the reasoning step, tool call, and observation at turn t , respectively; and $\hat{y}_\tau$ is the final answer.

An instructive trajectory must satisfy two complementary requirements. **Outcome Validity** requires the trajectory to terminate in a correct answer under the task-specific evaluator. However, correctness alone does not make it instructive: a strong teacher may infer the answer from parametric knowledge or reasoning shortcuts while still invoking tools because the synthesis prompt encourages it. Such a trajectory is successful in outcome but spurious as supervision: it teaches that tool calls co-occur with correct answers, without establishing that the tool interactions are useful.

Causal Utility therefore requires the recorded tool calls and observations to improve a fixed probe model’s answer reliability relative to a no-tool condition, while the original image and query remain unchanged. Sections 3.2 and 3.4

operationalize these requirements through repeated stochastic probe trials. Figure 2 turns them into a three-stage pipeline. **Difficulty Screening** first selects queries that the probe model cannot reliably solve from the original image alone. **Domain-Specific Trajectory Synthesis** then uses domain-specific tool-use patterns to guide a stronger teacher toward acquiring evidence suited to each domain’s visual bottleneck. Finally, **Supervision Verification** evaluates outcome validity and estimates causal utility by comparing the probe model under tool-trace-conditioned and no-tool conditions. Only trajectories satisfying both criteria are retained to form OpenVisTool-42K.

3.2 Difficulty Screening

We collect candidate instances from five domains: Chart, Table, GUI Grounding, Visual Search, and Web-to-HTML. Since many queries can already be solved from the original image, synthesizing tool-use trajectories for them may introduce redundant tool calls. We therefore screen for queries that a fixed probe model cannot reliably answer without tools. Let $x = (I, q, y^*, d)$ denote a candidate instance, where d is the task domain, and let $E_d(\hat{y}, y^*) \in \{0, 1\}$ denote the corresponding domain-specific evaluator. For each x , we run a fixed probe model π_0 for K independent no-tool trials. Let $Y_{\pi_0}^{(k)}(I, q)$ denote its answer in trial k . We compute the empirical success rate

$$\bar{p}_0(x) = \frac{1}{K} \sum_{k=1}^K E_d(Y_{\pi_0}^{(k)}(I, q), y^*). \quad (2)$$

For difficulty threshold γ , we retain an instance if $\bar{p}_0(x) \leq \gamma$. We use Qwen3.5-9B (Qwen Team 2026) as π_0 and set $K = 4$, $\gamma = 0.5$, retaining queries answered correctly in at most two trials. Dataset sources and preprocessing details are provided in Appendix A.

3.3 Domain-Specific Trajectory Synthesis

For each instance retained by Difficulty Screening, we use Qwen3.5-Plus as a teacher model π_{teacher} to synthesize a candidate trajectory with a shared visual toolset. Let h_t denote the interaction history before turn t . The rollout proceeds as

$$\begin{aligned} (r_t, c_t) &\sim \pi_{\text{teacher}}(\cdot \mid h_t, P_d), \\ o_t &= \text{Exec}(c_t), \\ h_{t+1} &= h_t \oplus (r_t, c_t, o_t), \end{aligned} \quad (3)$$

where P_d is the tool-use pattern associated with domain d . The process continues until the teacher produces a final answer, and the complete sequence of reasoning steps, tool calls, and observations is preserved.

A generic instruction to “use tools when helpful” may elicit arbitrary or decorative calls. We instead define P_d around the dominant visual bottleneck of each domain: series isolation, comparison, and computation for Chart; relevant-cell localization and row-column alignment for Table; progressive target localization and verification for GUI Grounding and Visual Search; and iterative render-compare-revise for Web-to-HTML. All domains use the same underlying tool interface. Consequently, the synthesized trajectories emphasize reusable evidence-acquisition behaviors rather than domain-specific APIs. The full toolset and domain-specific instructions are provided in Appendices B and C, respectively.

3.4 Supervision Verification

We first discard malformed trajectories, including sessions with missing observations, failed tool executions, or invalid file references. For each remaining trajectory, we verify **outcome validity** using the domain-specific evaluator E_d . Only trajectories satisfying $E_d(\hat{y}_\tau, y^*) = 1$ proceed to the causal-utility test. Depending on the domain, E_d uses answer matching, point-in-box evaluation, or rendered-page comparison with a VLM judge.

We then evaluate **causal utility**. For each outcome-valid trajectory, let $Z_\tau = ((c_t, o_t))_{t=1}^T$ denote the sequence of tool calls and corresponding observations supplied, together with the original image and query, to the same probe model π_0 used in Difficulty Screening. The teacher’s reasoning and final answer are excluded to prevent solution leakage. Let $Y_{\pi_0}^{(k)}(I, q; Z_\tau)$ denote the probe’s answer in tool-trace-conditioned trial k . Using the same $K = 4$ independent trials as in Difficulty Screening, we compute

$$\bar{p}_{\text{tool}}(x, \tau) = \frac{1}{K} \sum_{k=1}^K E_d \left(Y_{\pi_0}^{(k)}(I, q; Z_\tau), y^* \right). \quad (4)$$

For utility threshold δ , a trajectory passes Supervision Verification when both

$$\begin{aligned} E_d(\hat{y}_\tau, y^*) &= 1, \\ \bar{p}_{\text{tool}}(x, \tau) - \bar{p}_0(x) &\geq \delta \end{aligned} \quad (5)$$

hold. We set $\delta = 0.25$. Reusing the same probe model and evaluator makes $\bar{p}_{\text{tool}}(x, \tau)$ directly comparable with $\bar{p}_0(x)$. Together, the conditions in Eq. 5 retain only outcome-correct trajectories whose visual evidence measurably improves answer reliability. Each verified trajectory forms a supervised fine-tuning sample that preserves the teacher’s reasoning, tool calls, observations, and final answer, providing process-level supervision for when and how to use visual evidence. Domain-wise statistics for OpenVisTool-42K and qualitative examples are provided in Appendices A and D, respectively.

4 Experiments

We design experiments to validate our central claim—that effective visual tool use is learned from instructive trajectories rather than merely successful ones—and to answer the following questions:

- **Effectiveness:** Does fine-tuning on OpenVisTool-42K teach models to effectively leverage visual tools, and can it lift open-source models to closed-source performance levels? (§4.3)
- **Generalization:** Does the learned tool-use capability transfer to out-of-distribution visual tasks unseen during training, rather than overfitting to domain-specific tool-calling patterns? (§4.4)
- **Supervision verification:** Are outcome validity and causal utility both necessary for constructing effective visual tool-use supervision? (§4.5)
- **Cross-domain synergy:** Does visual tool use emerge as a transferable meta-skill across domains, and does multi-domain training resolve strategy conflicts that single-domain training introduces? (§4.6)

4.1 OpenVisTool-Bench: Isolating Tool-Use Ability

General-purpose visual benchmarks often mix tool-relevant instances with questions that can already be solved from the initial image encoding. OpenVisTool-Bench instead isolates the incremental value of active visual evidence acquisition by focusing on instances where operations such as cropping, enhancement, or structure detection materially improve task performance. Whereas Agentic-MME (Wei et al. 2026) evaluates broad multimodal agency and VTC-Bench (Zhu et al. 2026) stresses compositional tool execution, OpenVisTool-Bench specifically measures the performance gain enabled by visual-tool access.

Construction. We construct each domain according to whether its source benchmark already targets tool-demanding tasks.

- **Chart and Table.** We source Chart instances from CharXiv (Wang et al. 2024b) and ChartMuseum (Tang et al. 2025), and Table instances from TableVQA-Bench (Kim, Yim, and Song 2024) and MMTBench (Titiya et al. 2026). For each instance, three strong models (GPT-5.4, Gemini-3.0-Flash, and Qwen3.5-Plus) are evaluated with and without tools using five trials per condition. We retain the instance if at least one model achieves a tool-use gain of $\text{avg@5}_{\text{tool}} - \text{avg@5}_{\text{no-tool}} \geq 0.4$.

Model	Setting	OpenVisTool-Bench					
		Chart	GUI	Table	VisualSearch	Web2HTML	AVG
GPT-5.5	w/o tool	63.3	17.7	48.9	27.7	51.7	41.9
	with tool	66.6	24.2	53.9	46.2	54.3	<u>49.0</u>
Kimi K2.6	w/o tool	50.9	17.1	34.4	13.8	49.4	33.1
	with tool	<u>64.3</u>	18.8	<u>51.6</u>	49.8	<u>53.6</u>	47.6
Qwen2.5-VL-7B	w/o tool	18.2	5.1	21.2	28.3	23.1	19.2
Thyme	with tool	19.8	0.0	17.9	39.2	14.3	18.2
DeepEyes V2	with tool	20.9	1.7	20.4	31.8	2.4	15.5
Qwen3.5-4B	w/o tool	31.6	22.0	25.4	34.0	40.8	30.8
	with tool	32.5	23.5	32.3	39.6	41.6	33.9
	+OpenVisTool-42K with tool	43.0 (+11.4)	27.4 (+5.4)	37.3 (+11.9)	56.6 (+22.6)	43.4 (+2.6)	41.5 (+10.7)
Qwen3-VL-8B-Instruct	w/o tool	22.1	13.0	26.7	31.8	26.5	24.0
	with tool	20.9	12.6	24.6	30.2	28.1	23.3
	+OpenVisTool-42K with tool	26.1 (+4.0)	19.7 (+6.7)	35.3 (+8.6)	50.7 (+18.9)	32.9 (+6.4)	32.9 (+8.9)
Qwen3.5-9B	w/o tool	34.6	27.1	29.0	35.6	40.1	33.3
	with tool	28.4	24.8	26.8	38.7	42.0	32.1
	+OpenVisTool-42K with tool	49.3 (+14.7)	31.6 (+4.5)	43.1 (+14.1)	59.4 (+23.8)	45.5 (+5.4)	45.8 (+12.5)
Qwen3.5-27B	w/o tool	45.7	28.9	29.8	43.2	46.6	38.8
	with tool	47.7	29.1	43.3	54.7	44.3	43.8
	+OpenVisTool-42K with tool	60.7 (+15.0)	31.6 (+2.7)	48.2 (+18.4)	<u>57.3</u> (+14.1)	48.7 (+2.1)	49.3 (+10.5)

Table 1: Main results on OpenVisTool-Bench under the avg@4 evaluation protocol. “w/o tool” and “with tool” denote evaluation without and with access to the corresponding visual tool environment, respectively. Rows marked with +OpenVisTool-42K denote models fine-tuned on our training data, and green $+x$ suffixes report absolute gains over the “w/o tool” result of the same backbone. **Bold** and underlined entries indicate the best and second-best results in each column, respectively.

• GUI Grounding, Visual Search, and Web-to-HTML.

These sources already require active visual interaction—precise localization, fine-grained retrieval, and iterative rendering—so we apply no additional tool-gain filtering. We use the 117 ScreenSpot-Pro (Li et al. 2025a) instances with the smallest target regions, together with VisualProbe-Hard (Lai et al. 2026) and Vision2Web-Level1 (He et al. 2026).

More details on the construction of OpenVisTool-Bench, including source-specific sampling and filtering procedures, are provided in Appendix E.

Evaluation protocol. Each domain uses a task-specific evaluator. Chart, Table, and Visual Search use LLM-as-a-judge for binary correctness. GUI uses rule-based point-in-box accuracy. Web-to-HTML follows the official Vision2Web evaluation protocol, measuring component-level visual fidelity between the rendered output and reference page on a 0–100 scale. We report each domain score and the macro-average across domains under the avg@4 protocol.

4.2 Experimental Setup

Benchmarks. We primarily evaluate on OpenVisTool-Bench. To assess generalization, we additionally test on Agentic-MME and VTC-Bench. We also report results on the full Chart and Table source benchmarks in Appendix F to confirm that gains are not artifacts of sample selection.

Baselines. We compare three types of models. (i) General-purpose frontier models: GPT-5.5 and Kimi K2.6 (Kimi Team 2026), both evaluated without tools and with our shared toolset. (ii) Smaller-scale general-purpose models: Qwen2.5-VL-7B (Bai et al. 2025b), Qwen3-VL-8B-Instruct (Bai et al. 2025a) and Qwen3.5 (Qwen Team 2026) models are evaluated both without tools and with our shared toolset before fine-tuning. (iii) Open-source visual tool-use agents: Thyme (Zhang et al. 2026) and DeepEyes V2 (Hong et al. 2026), evaluated with the code-based tool environments used in their original work rather than our toolset.

Training. We fine-tune Qwen3.5-4B/9B/27B and Qwen3-VL-8B-Instruct on OpenVisTool-42K for 3 epochs using SWIFT (Zhao et al. 2025b). More implementation details are provided in Appendix G.

4.3 Main Results

Table 1 summarizes performance across all models. We highlight several key findings below.

Consistent gains across all base models and domains. Our method yields substantial improvements over the no-tool baselines across all domains and model scales. Across models ranging from 4B to 27B, fine-tuning with OpenVisTool-42K brings an average gain of 10.7 points on OpenVisTool-Bench, with the most pronounced improvement observed in VisualSearch (+23.8 points). Notably, even without any fine-

Model	Agentic-MME	VTC-Bench
Qwen3.5-4B	29.9	27.5
+OpenVisTool-42K	34.5 (+4.6)	38.8 (+11.3)
Qwen3-VL-8B-Instruct	25.6	28.2
+OpenVisTool-42K	32.3 (+6.7)	29.1 (+0.9)
Qwen3.5-9B	32.7	30.1
+OpenVisTool-42K	41.6 (+8.9)	41.4 (+11.3)
Qwen3.5-72B	37.1	34.9
+OpenVisTool-42K	41.1 (+4.0)	45.2 (+10.3)

Table 2: Out-of-distribution results on Agentic-MME and VTC-Bench. Base models are evaluated without tools, while models trained on OpenVisTool-42K are evaluated with tools. Green suffixes show absolute gains over the corresponding no-tool backbone.

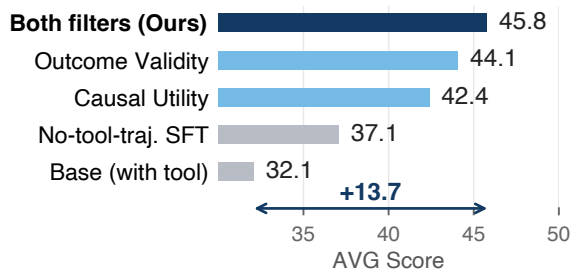


Figure 3: Ablation of the criteria for instructive tool-use supervision on Qwen3.5-9B under a fixed training budget. Requiring both outcome validity and causal utility achieves the best performance, outperforming either criterion alone and the no-tool-trajectory SFT baseline.

tuning, simply equipping strong off-the-shelf models with our toolset provides significant benefits: GPT-5.5 and Kimi K2.6 achieve 7.1 and 14.5 absolute gains, respectively, demonstrating the inherent utility of the designed tools. However, smaller open-source models derive little or even negative benefit from tool access without dedicated training. Fine-tuning on OpenVisTool-42K enables them to use tools effectively, bringing their performance close to that of closed-source models. For example, Qwen3.5-9B with our training achieves an average of 45.8, surpassing GPT-5.5’s 41.9 without tools and closely approaching its tool-augmented performance of 49.0. In contrast, existing tool-use models Thyme and DeepEyes V2, trained on narrow single-domain data, perform worse on the five-domain average than vanilla base models *without any tool access*—they improve on their specialty but collapse catastrophically elsewhere. This comparison highlights that a diverse, multi-domain training corpus, as provided by OpenVisTool-42K, is essential for robust and generalizable visual tool use.

Largest gains emerge where tools unlock new visual evidence. For domains where the answer is largely derivable from surface-level visual content—such as Chart, which primarily tests text and semantic extraction from images—a

Trajectory source	OpenVisTool-Bench	Agentic-MME	VTC-Bench
None (Qwen3.5-9B w/o tool)	33.3	32.7	30.1
Correctness-only model	42.1	36.5	36.5
Instructive model (ours)	44.5	37.7	38.1

Table 3: Trajectory replay analysis using the original Qwen3.5-9B probe model. The probe model answers each query conditioned on the tool call arguments and returned observations generated by the correctness-only or instructively trained model. The generator’s reasoning and final answer are excluded. “None” denotes evaluation without tool access or trajectory replay.

sufficiently capable model can already perform well without tools (e.g., GPT-5.5 scores 63.3 on Chart without tools). However, for more challenging domains involving dense details and hidden evidence—such as VisualSearch, which demands fine-grained spatial and geometric reasoning—models often struggle to succeed with their inherent visual perception alone. In these scenarios, invoking auxiliary tools to crop, zoom, or reformat visual content becomes essential. This explains why our method yields the most substantial gains precisely in these demanding domains (e.g., Qwen3.5-9B: +23.8 points on VisualSearch), where tools effectively bridge the gap between what the model can perceive and what the task requires.

4.4 Out-of-Distribution Generalization

To test whether the tool-use capability learned from our five training domains transfers beyond them, we further evaluate on Agentic-MME and VTC-Bench with out-of-distribution domains held out from training. Table 2 shows that, across all four backbones, models trained on OpenVisTool-42K consistently outperform their corresponding no-tool backbones on both Agentic-MME and VTC-Bench. This consistent improvement on benchmarks outside the five training domains suggests that OpenVisTool-42K teaches transferable visual evidence-acquisition behavior rather than merely encouraging domain-specific tool-calling patterns.

4.5 Ablation Study

To validate that effective visual tool-use learning requires both correct answers and causally useful tool traces, we compare four Qwen3.5-9B supervision variants under the same training budget. The first three use tool trajectories retained by (1) both outcome validity and causal utility (full OpenVisTool-42K), (2) outcome validity alone, or (3) causal utility alone. The fourth is a no-tool-trajectory SFT baseline, constructed by re-synthesizing trajectories without tool invocation for the queries in the full variant.

Gains originate from tool-use, not from SFT alone. As shown in Figure 3, every tool-augmented variant outperforms the no-tool-trajectory SFT baseline, confirming that improvements stem from learned tool invocation rather than simply training on additional difficult queries.

Variant	Setting	Chart	GUI	Table	VisualSearch	Web2HTML	AVG
Qwen3.5-9B	w/o tool	34.6	27.1	29.0	35.6	40.1	33.3
	with tool	28.4	24.8	26.8	38.7	42.0	32.1
+ OpenVisTool-42K (full)	with tool	49.3	<u>31.6</u>	43.1	59.4	45.5	45.8
<i>Single-domain slices</i>							
+ Chart	with tool	<u>46.6</u>	22.2	37.1	44.3	36.5	37.4
+ GUI	with tool	41.4	36.3	38.7	52.8	39.1	<u>41.7</u>
+ Table	with tool	45.7	20.5	38.9	45.8	39.2	38.0
+ VisualSearch	with tool	45.7	23.3	<u>40.9</u>	<u>57.3</u>	21.6	37.8
+ Web2HTML	with tool	41.6	18.2	38.5	42.2	<u>44.6</u>	37.0

Table 4: Cross-domain transfer analysis on OpenVisTool-Bench with Qwen3.5-9B as base model. We fine-tune the model on either the full OpenVisTool-42K mixture or an individual domain slice and evaluate all fine-tuned variants with tools across the five domains. **Bold** and underlined entries indicate the best and second-best results in each column, respectively.

Both filters are complementary and necessary. Intersecting both filters consistently dominates either filter in isolation, with outcome-validity-only and causal-utility-only trailing by 1.7 and 3.4 points, respectively. The two filters address distinct failure modes: outcome validity removes demonstrations that lead to incorrect answers and would teach erroneous behavior, while causal utility discards trajectories where tools are invoked but their observations contribute no measurable benefit to reaching the solution. Their combination yields the highest-quality training signal.

Instructive filtering yields more useful tool interactions. To probe the learned behavior beyond the task accuracy of the fine-tuned models themselves, we conduct a trajectory replay analysis using the original Qwen3.5-9B probe model. Specifically, we replay the tool-interaction trajectories generated by fine-tuned models. The replayed context includes only the tool call arguments and their corresponding returned observations. We deliberately exclude all intermediate reasoning and final answers, which prevents the probe model from directly copying the generator’s answer or simply imitating its rationale. As shown in Table 3, trajectories from both trained models improve the probe model, but those from the model trained on instructive trajectories consistently outperform correctness-only trajectories. This behavior-level transfer suggests that causal-utility filtering induces an evidence-acquisition policy whose resulting interactions are more useful to another model, rather than merely producing tool calls that accompany successful outcomes.

4.6 Cross-Domain Transfer

We fine-tune Qwen3.5-9B on each single-domain slice of OpenVisTool-42K separately and evaluate on all five domains. Full results are reported in Table 4.

Tool-use transfers as a domain-general meta-skill. Training on any single domain improves overall performance over the tool-enabled base model, with gains often extending to domains unseen during training. Most notably, GUI-only training improves VisualSearch by 14.1 points. This cross-domain benefit suggests that the model learns reusable tool-use primitives, such as localized cropping and spatial grounding, rather than only domain-specific solutions.

Transfer is asymmetric, with both positive and negative effects. The transfer is not uniformly beneficial because different domains reward different interaction patterns. For example, VisualSearch-only training lowers Web2HTML performance from 42.0 to 21.6. We attribute this degradation to a mismatch in tool-use strategies: repeatedly cropping small regions is effective for fine-grained search but conflicts with global, full-page reasoning. Single-domain training can therefore over-specialize the tool-use policy and harm domains that require incompatible strategies.

The full mixture resolves strategy conflicts. The full OpenVisTool-42K mixture achieves a 45.8 average, surpassing all single-domain variants, and performs best on four of the five domains. Exposure to diverse interaction patterns helps the model resolve strategy conflicts through *context-conditional* tool use: it learns not only how to invoke tools, but also which strategy is appropriate for the visual input. GUI is the only exception: its fixed localization protocol and point-in-box metric favor specialized coordinate calibration, allowing GUI-only training to outperform the full mixture. Nevertheless, the full mixture still substantially improves over the tool-enabled base model, demonstrating effective cross-domain transfer.

5 Conclusion

In this work, we introduce OpenVisTool, a framework for constructing **instructive** visual tool-use trajectories that addresses a limitation of outcome-only filtering: an answer-correct trajectory may contain tool observations that do not contribute to its answer. OpenVisTool defines instructive supervision through outcome validity and causal utility, and operationalizes these criteria with difficulty screening, domain-specific trajectory synthesis, and supervision verification to construct OpenVisTool-42K across five domains. We separately build OpenVisTool-Bench across the same domains to evaluate models’ ability to acquire and use visual evidence through tools. Across four backbones, training on OpenVisTool-42K consistently improves performance. Trajectory replay and cross-domain results further suggest that instructive supervision promotes useful evidence acquisition and transferable tool-use behavior.

References

- Alonso, I.; Miranda, I.; Agirre, E.; and Lapata, M. 2026. TABLET: A Large-Scale Dataset for Robust Visual Table Understanding. In *The Fourteenth International Conference on Learning Representations*.
- Bai, S.; Cai, Y.; Chen, R.; Chen, K.; Chen, X.; Cheng, Z.; Deng, L.; Ding, W.; Gao, C.; Ge, C.; Ge, W.; Guo, Z.; Huang, Q.; Huang, J.; Huang, F.; Hui, B.; Jiang, S.; Li, Z.; Li, M.; Li, M.; Li, K.; Lin, Z.; Lin, J.; Liu, X.; Liu, J.; Liu, C.; Liu, Y.; Liu, D.; Liu, S.; Lu, D.; Luo, R.; Lv, C.; Men, R.; Meng, L.; Ren, X.; Ren, X.; Song, S.; Sun, Y.; Tang, J.; Tu, J.; Wan, J.; Wang, P.; Wang, P.; Wang, Q.; Wang, Y.; Xie, T.; Xu, Y.; Xu, H.; Xu, J.; Yang, Z.; Yang, M.; Yang, J.; Yang, A.; Yu, B.; Zhang, F.; Zhang, H.; Zhang, X.; Zheng, B.; Zhong, H.; Zhou, J.; Zhou, F.; Zhou, J.; Zhu, Y.; and Zhu, K. 2025a. Qwen3-VL Technical Report. arXiv:2511.21631.
- Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; Wang, S.; Tang, J.; Zhong, H.; Zhu, Y.; Yang, M.; Li, Z.; Wan, J.; Wang, P.; Ding, W.; Fu, Z.; Xu, Y.; Ye, J.; Zhang, X.; Xie, T.; Cheng, Z.; Zhang, H.; Yang, Z.; Xu, H.; and Lin, J. 2025b. Qwen2.5-VL Technical Report. arXiv:2502.13923.
- Feng, J.; Huang, S.; Qu, X.; Zhang, G.; Qin, Y.; Zhong, B.; Jiang, C.; Chi, J.; and Zhong, W. 2025. ReTool: Reinforcement Learning for Strategic Tool Use in LLMs. arXiv:2504.11536.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2025. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. In *The Thirteenth International Conference on Learning Representations*.
- Guo, Z.; Hong, M.; Zhang, F.; Jia, K.; and Jin, T. 2025. Thinking with Programming Vision: Towards a Unified View for Thinking with Images. arXiv:2512.03746.
- He, Z.; Hong, W.; Yang, Z.; Pan, Z.; Liu, M.; Gu, X.; and Tang, J. 2026. Vision2Web: A Hierarchical Benchmark for Visual Website Development with Agent Verification. In *Forty-third International Conference on Machine Learning*.
- Hong, J.; Zhao, C.; Zhu, C.; Lu, W.; Xu, G.; and XingYu. 2026. DeepEyesV2: Toward Agentic Multimodal Model. In *The Fourteenth International Conference on Learning Representations*.
- Hou, X.; Xu, S.; Biyani, M.; Li, M.; Liu, J.; Hollon, T. C.; and Wang, B. 2025. CodeV: Code with Images for Faithful Visual Reasoning via Tool-Aware Policy Optimization. arXiv:2511.19661.
- Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S. O.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *Second Conference on Language Modeling*.
- Kaur, R.; Srishankar, N.; Zeng, Z.; and Ganesh, S. 2026. ChartAgent: A Multimodal Agent for Visually Grounded Reasoning in Complex Chart Question Answering. In Liakata, M.; Moreira, V. P.; Zhang, J.; and Jurgens, D., eds., *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*, 18488–18555. Association for Computational Linguistics.
- Kim, Y.; Yim, M.; and Song, K. Y. 2024. TableVQA-Bench: A Visual Question Answering Benchmark on Multiple Table Domains. arXiv:2404.19205.
- Kimi Team. 2026. Kimi K2.6: Advancing Open-Source Coding.
- Lai, X.; Li, J.; Li, W.; Liu, T.; Li, T.; and Zhao, H. 2026. Mini-o3: Scaling Up Reasoning Patterns and Interaction Turns for Visual Search. In *The Fourteenth International Conference on Learning Representations*.
- Li, K.; Ziyang, M.; Lin, H.; Luo, Z.; Tian, Y.; Ma, J.; Huang, Z.; and Chua, T.-S. 2025a. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. In *Workshop on Reasoning and Planning for Large Language Models*.
- Li, X.; Dong, G.; Jin, J.; Zhang, Y.; Zhou, Y.; Zhu, Y.; Zhang, P.; and Dou, Z. 2025b. Search-o1: Agentic Search-Enhanced Large Reasoning Models. arXiv:2501.05366.
- Li, X.; Jin, J.; Dong, G.; Qian, H.; Wu, Y.; Wen, J.-R.; Zhu, Y.; and Dou, Z. 2025c. WebThinker: Empowering Large Reasoning Models with Deep Research Capability. arXiv:2504.21776.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, 39578–39601.
- Liu, Z.; Lin, H.; Qin, C.; Wang, X.; Gao, X.; Li, Y.; Cai, M.; Zhu, Y.; Zhong, Z.; Pei, Q.; Pan, Z.; Shang, X.; Cui, B.; He, C.; Zhang, W.; and Wu, L. 2026. ChartVerse: Scaling Chart Reasoning via Reliable Programmatic Synthesis from Scratch. arXiv:2601.13606.
- OpenAI. 2025. Thinking with images.
- Qwen Team. 2026. Qwen3.5: Accelerating Productivity with Native Multimodal Agents.
- Sarch, G.; Cai, L.; Wang, Q.; Wu, H.; Chen, D.; and Liu, Z. 2026. Vero: An Open RL Recipe for General Visual Reasoning. arXiv:2604.04917.
- Su, A.; Wang, H.; Ren, W.; Lin, F.; and Chen, W. 2025a. Pixel Reasoner: Incentivizing Pixel Space Reasoning via Curiosity-Driven Reinforcement Learning. In Belgrave, D.; Zhang, C.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi, M.; and Chen, N., eds., *Advances in Neural Information Processing Systems*, volume 38, 8222–8251. Curran Associates, Inc.
- Su, Z.; Li, L.; Song, M.; Hao, Y.; Yang, Z.; Zhang, J.; Chen, G.; Gu, J.; Li, J.; Qu, X.; and Cheng, Y. 2025b. OpenThinkIMG: Learning to Think with Images via Visual Tool Reinforcement Learning. arXiv:2505.08617.
- Su, Z.; Xia, P.; Guo, H.; Liu, Z.; Ma, Y.; Qu, X.; Liu, J.; Li, Y.; Zeng, K.; Yang, Z.; Li, L.; Cheng, Y.; Ji, H.; He, J.; and Fung, Y. R. 2025c. Thinking with Images for Multimodal Reasoning: Foundations, Methods, and Future Frontiers. arXiv:2506.23918.
- Tang, L.; Kim, G.; Zhao, X.; Lake, T.; Ding, W.; Yin, F.; Singhal, P.; Wadhwa, M.; Liu, Z. L.; Sprague, Z.; Namuduri, R.; Hu, B.; Rodriguez, J. D.; Peng, P.; and Durrett, G. 2025. ChartMuseum: Testing Visual Reasoning Capabilities of Large Vision-Language Models. In Belgrave, D.; Zhang, C.; Montoya, L. N.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi,

- M.; Chen, N.; Ruíz, I. V. M.; and Loaiza-Bonilla, A., eds., *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.
- Titia, P. Y.; Trivedi, J.; Baral, C.; and Gupta, V. 2026. MMTabReal: Real-World Benchmark for Multimodal Table Understanding. In Liakata, M.; Moreira, V. P.; Zhang, J.; and Jurgens, D., eds., *Findings of the Association for Computational Linguistics: ACL 2026*, 41156–41176. San Diego, California, United States: Association for Computational Linguistics. ISBN 979-8-89176-395-1.
- Wang, P.; Li, L.; Shao, Z.; Xu, R.; Dai, D.; Li, Y.; Chen, D.; Wu, Y.; and Sui, Z. 2024a. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. In Ku, L.; Martins, A.; and Srikumar, V., eds., *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, 9426–9439. Association for Computational Linguistics.
- Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; Wang, J.; Deng, J.; Guo, X.; Xu, Y.; Wu, C. H.; Shen, Z.; Li, Z.; Li, R.; Li, X.; Chen, J.; Zheng, B.; Li, P.; Lei, F.; Cao, R.; Fu, Y.; Shin, D.; Shin, M.; Hu, J.; Wang, Y.; Chen, J.; Ye, Y.; Zhang, D.; Wang, Y.; Wang, H.; Yang, D.; Zhong, V.; Charles, Y.; Yang, Z.; and Yu, T. 2025. OpenCUA: Open Foundations for Computer-Use Agents. In Belgrave, D.; Zhang, C.; Montoya, L. N.; Lin, H.; Pascanu, R.; Koniusz, P.; Ghassemi, M.; Chen, N.; Ruíz, I. V. M.; and Loaiza-Bonilla, A., eds., *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.
- Wang, Z.; Xia, M.; He, L.; Chen, H.; Liu, Y.; Zhu, R.; Liang, K.; Wu, X.; Liu, H.; Malladi, S.; Chevalier, A.; Arora, S.; and Chen, D. 2024b. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. In Globersons, A.; Mackey, L.; Belgrave, D.; Fan, A.; Paquet, U.; Tomczak, J. M.; and Zhang, C., eds., *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- Wei, Q.; Yang, Y.; Wang, S.; Chen, J.; Wang, B.; Wang, J.; Chen, S.; Li, Z.; Shi, Y.; Tang, Y.; Wang, W.; Yu, Y.; Fu, C.; Li, Q.; and Zhang, Y.-F. 2026. Agentic-MME: What Agentic Capability Really Brings to Multimodal Intelligence? arXiv:2604.03016.
- Wu, M.; Yang, J.; Jiang, J.; Li, M.; Yan, K.; Yu, H.; Zhang, M.; Zhai, C.; and Nahrstedt, K. 2026. VTool-R1: VLMs Learn to Think with Images via Reinforcement Learning on Multimodal Tool Use. In *The Fourteenth International Conference on Learning Representations*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; and Qiao, Y. 2025. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *The Thirteenth International Conference on Learning Representations*.
- Xue, Z.; Zheng, L.; Liu, Q.; Li, Y.; Zheng, X.; Ma, Z.; and An, B. 2025. SimpleTIR: End-to-End Reinforcement Learning for Multi-Turn Tool-Integrated Reasoning. arXiv:2509.02479.
- Yang, Y.; Patel, A.; Deitke, M.; Gupta, T.; Weihs, L.; Head, A.; Yatskar, M.; Callison-Burch, C.; Krishna, R.; Kembhavi, A.; and Clark, C. 2025. Scaling Text-Rich Image Understanding via Code-Guided Synthetic Multimodal Data Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 17486–17505.
- Yang, Z.; Hong, W.; Xu, M.; Fan, X.; Wang, W.; Cheng, J.; Gu, X.; and Tang, J. 2026. UI2Code[^]N\$: A Visual Language Model for Test-Time Scalable Interactive UI-to-Code Generation.
- Yu, L.; Jiang, W.; Shi, H.; Yu, J.; Liu, Z.; Zhang, Y.; Kwok, J.; Li, Z.; Weller, A.; and Liu, W. 2024. Metamath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations*, volume 2024, 45040–45061.
- Yuan, Z.; Yuan, H.; Li, C.; Dong, G.; Lu, K.; Tan, C.; Zhou, C.; and Zhou, J. 2023. Scaling relationship on learning mathematical reasoning with large language models. arXiv:2308.01825.
- Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. D. 2022. STaR: Bootstrapping Reasoning With Reasoning. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Zhang, Y.; Hu, L.; Sun, H.; Wang, P.; Wei, Y.; Yin, S.; Pei, J.; Shen, W.; Xia, P.; Peng, Y.; Xie, T.; Li, E.; Liu, Y.; Song, X.; and Zhou, Y. 2025. Skywork-R1V4: Toward Agentic Multimodal Intelligence through Interleaved Thinking with Images and DeepResearch. arXiv:2512.02395.
- Zhang, Y.; Lu, X.; Yin, S.; Fu, C.; Chen, W.; Hu, X.; Wen, B.; Jiang, K.; Liu, C.; Zhang, T.; fan, H.; Chen, K.; Chen, J.; Ding, H.; Tang, K.; Zhang, Z.; Wang, L.; Yang, F.; Gao, T.; and Zhou, G. 2026. Thyme: Think Beyond Images. In *The Fourteenth International Conference on Learning Representations*.
- Zhao, X.; Jiang, D.; Zeng, Z.; Chen, L.; Qiu, H.; Huang, J.; Zhong, Y.; Zheng, L.; Cao, Y.; and Ma, L. 2025a. VinciCoder: Unifying Multimodal Code Generation via Coarse-to-fine Visual Reinforcement Learning. arXiv:2511.00391.
- Zhao, X.; Tan, Z.; Sheng, D.; Chen, T.; Liu, Y.; Wu, Y.; Gong, T.; Chu, Q.; and Yu, N. 2026. Learning to Focus and Precise Cropping: A Reinforcement Learning Framework with Information Gaps and Grounding Loss for MLLMs. arXiv:2603.27494.
- Zhao, Y.; Huang, J.; Hu, J.; Wang, X.; Mao, Y.; Zhang, D.; Jiang, Z.; Wu, Z.; Ai, B.; Wang, A.; Zhou, W.; and Chen, Y. 2025b. SWIFT: A Scalable Lightweight Infrastructure for Fine-Tuning. In Walsh, T.; Shah, J.; and Kolter, Z., eds., *Thirty-Ninth AAAI Conference on Artificial Intelligence, Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence, Fifteenth Symposium on Educational Advances in Artificial Intelligence, AAAI 2025, Philadelphia*,

PA, USA, February 25 - March 4, 2025, 29733–29735. AAAI Press.

Zheng, Z.; Yang, M.; Hong, J.; Zhao, C.; Xu, G.; Yang, L.; Shen, C.; and XingYu. 2026. DeepEyes: Incentivizing “Thinking with Images” via Reinforcement Learning. In *The Fourteenth International Conference on Learning Representations*.

Zhu, X.; Dong, Y.; Wang, R.; Shi, Y.; Wu, Z.; Peng, Y.; Zhang, Y.; Lou, Y.; Zhang, Y.; Liu, Z.; Bai, Y.; and Zhou, Y. 2026. VTC-Bench: Evaluating Agentic Multimodal Models via Compositional Visual Tool Chaining. arXiv:2603.15030.

Supplementary Material

A OpenVisTool-42K Construction Details

A.1 Source Datasets and Preprocessing

For each example, the task image and query serve as the task input to the teacher, whose rollout is additionally conditioned on the domain-specific tool-use instructions detailed in Section C. The reference answer is withheld from the teacher and used only offline for source preprocessing where applicable, difficulty screening, and outcome-validity verification. We do not use source-provided reasoning traces or tool-use trajectories as supervision. We first apply the domain-specific preprocessing described below, after which all candidate pools undergo the uniform difficulty screening introduced in Section 3.2 of the main paper.

Chart. We use 100,000 examples from ChartVerse-SFT (Liu et al. 2026) without additional preprocessing.

Table. We draw from CoSyn (Yang et al. 2025) and TABLET (Alonso et al. 2026). From CoSyn, we select the table-image subset and discard examples with an empty or invalid question or answer. From TABLET, we use examples from the HiTab, TabMWP, TAT-QA, and WikiTQ training subsets. After preprocessing, the Table candidate pool contains 469,460 examples.

GUI Grounding. We draw from OS-Atlas (Wu et al. 2025), AgentNet (Wang et al. 2025), and UGround (Gou et al. 2025). From the Linux, macOS, and Windows portions of OS-Atlas, we discard annotations with missing instructions or images, malformed or non-normalized target boxes, low-resolution images (≤ 1 MP), or boxes covering at least 0.5% of the image. From AgentNet, we retain the high-resolution Ubuntu click subset. From UGround, we discard examples with empty, URL-like, or overly short instructions, invalid or overly large target boxes, low-resolution images, or non-landscape layouts, and then subsample the filtered pool. After preprocessing, the GUI Grounding candidate pool contains 698,422 examples.

Visual Search. We draw from Vero-600K (Sarch et al. 2026) and the DeepEyesV2-RL corpus (Hong et al. 2026). From Vero-600K, we select the PixelReasoner and VisualProbe components, yielding 9,744 examples. From DeepEyesV2-RL, we retain images whose shorter side is at least 768 pixels and remove chart-like examples using keyword rules followed by classification with Qwen3-VL-30B-A3B-Instruct, yielding 9,285 examples. The resulting Visual Search candidate pool contains 19,029 examples.

Web-to-HTML. We draw from VinciCoder (Zhao et al. 2025a). We remove screenshots dominated by a non-background color and construct a diverse pool spanning complex UIs, high-pixel-count screenshots, and typical 1280×720 layouts. We also replace the heterogeneous source prompts with a shared screenshot-to-HTML reconstruction instruction before teacher rollout. After preprocessing, the Web-to-HTML candidate pool contains 26,829 examples.

A.2 Trajectory Synthesis and Filtering Details

Teacher rollout. We synthesize trajectories with Qwen3.5-Plus in a function-calling agent loop. At each turn, the teacher produces a reasoning step and optional tool calls; each returned observation is appended to the context. Domain-specific instructions are injected through the rollout configuration, while all domains share the same toolset. Each query runs in an isolated workspace with its own media directory, preventing generated crops, annotations, masks, and rendered pages from colliding across examples. The loop terminates when the teacher returns a final answer or, for GUI Grounding, emits the terminal `computer_use` click.

Outcome validity and trajectory sanitation. After rollout, we first evaluate the teacher’s final output. Chart uses rule-based answer matching; Table and Visual Search use an LLM judge (Qwen3.5-27B); GUI Grounding uses point-in-box evaluation; and Web-to-HTML uses rendered-page comparison with a VLM judge. For Web-to-HTML, the generated HTML must be extractable and renderable and must receive a visual-consistency score of at least 80 from Qwen3.5-27B. We then reject trajectories with no tool call, no valid final response (or no terminal coordinate for GUI Grounding), more than 30 tool rounds, missing observations or generated images, failed tool executions, or invalid file references. Only trajectories that pass both outcome evaluation and these sanitation checks proceed to causal-utility filtering.

Causal-utility filtering. For each surviving trajectory, we provide the original image and query together with the recorded tool calls and responses, in chronological order, to the same Qwen3.5-9B probe used for difficulty screening. Generated images returned by tools are included with their corresponding responses, whereas the teacher’s reasoning and final answer are excluded to prevent answer leakage. We rerun the probe four times to compute the trajectory-conditioned `avg@4` and compare it with the corresponding no-tool `avg@4`. We retain the trajectory when the improvement is at least 0.1; together with the preceding outcome filter, this ensures that every retained trajectory satisfies both outcome validity and causal utility. Before constructing the context, each serialized tool call and its corresponding response are independently limited to 10,000 characters. The maximum generation lengths are 32,768 tokens for both difficulty screening and the causal-utility probe.

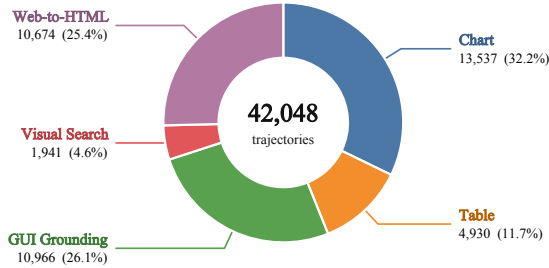
A.3 Final Dataset Statistics and Domain Distribution

The final training corpus contains 42,048 trajectories and 296,993 executed tool calls, averaging 7.06 calls per trajectory; every retained trajectory contains at least one tool invocation. Figure 4 (left) visualizes the domain composition. The distribution reflects the natural yield after the shared difficulty, outcome-validity, and causal-utility filters, without domain-level upsampling or rebalancing.

B The Complete Toolset

Our shared toolset consists of general-purpose tools and visual tools, summarized in Table 8.

OpenVisTool-42K



OpenVisTool-Bench

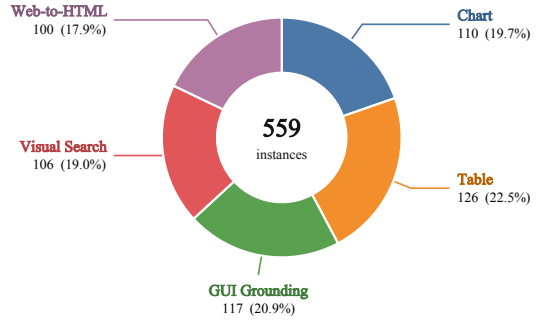


Figure 4: **Domain distributions of OpenVisTool-42K and OpenVisTool-Bench.** The 42,048 retained training trajectories reflect the natural yield of the shared filtering pipeline (left), while the 559 evaluation instances are approximately balanced across the same five domains (right).

General-Purpose Tools. The general-purpose tools provide file and system operations shared across all task domains.

Visual Tools. The visual tools are grouped by functionality. For GUI Grounding, we additionally use `computer_use` to express the final GUI action (e.g., `click`) in the required format without executing it.

C Domain-Specific Tool-use Instructions

Tables 9–13 provide the domain-specific instructions injected into the teacher’s system prompt during trajectory rollout.

D Qualitative Trajectory Examples

Figures 5–7 contrast two retained trajectories with an outcome-correct trajectory rejected by causal utility. We show only the decision-relevant reasoning and tool observations; the reported $\bar{p}_0$ and $\bar{p}_{\text{tool}}$ are the corresponding four-trial probe success rates (avg@4).

E OpenVisTool-Bench Construction Details

OpenVisTool-Bench contains 559 independently curated instances spanning the same five domains as the training corpus. Its domain composition is shown in the Figure 4 (right).

Chart and Table. The Chart source pool consists of 1,000 instances from the CharXiv validation set (Wang et al. 2024b) and 1,162 from ChartMuseum (Tang et al. 2025), while the Table source pool consists of 1,500 instances from TableVQA-Bench (Kim, Yim, and Song 2024) and 4,022 from MMT-Bench (Titiya et al. 2026). For both domains, we screen instances with GPT-5.4, Gemini-3.0-Flash, and Qwen3.5-Plus, running five trials with tools and five without tools. We retain the cross-model union of instances for which at least one model achieves $\text{avg@5}_{\text{tool}} - \text{avg@5}_{\text{no-tool}} > 0.4$. This yields 110 of the 2,162 Chart instances and, after restricting the TableVQA-Bench contribution to its VWTQ and VWTQ-Syn subsets, 126 of the 5,522 Table instances.

GUI Grounding. ScreenSpot-Pro (Li et al. 2025a) contains 1,581 instances. We retain the 117 instances whose relative target-box area falls between 5.70×10^{-5} and 8.43×10^{-5} of the screenshot area, without model-performance filtering.

Visual Search. We directly include all 106 instances from VisualProbe-Hard (Lai et al. 2026), which targets fine-grained exploratory visual search, without additional filtering.

Web-to-HTML. We use all 100 Level-1 static-webpage tasks from Vision2Web (He et al. 2026), where an agent reconstructs a responsive page from desktop, tablet, and mobile visual prototypes, without additional filtering.

F Additional Experimental Results

This section provides the full-benchmark evaluation referenced in the Experimental Setup of the main paper. Because OpenVisTool-Bench uses challenging subsets drawn from the Chart and Table source benchmarks, we additionally evaluate each backbone and its counterpart trained on OpenVisTool-42K on the complete, unfiltered source test sets. We disable tool invocation for this evaluation to isolate the capabilities transferred to the models themselves.

As shown in Table 5, training on OpenVisTool-42K improves every backbone on every full source benchmark. The pattern is consistent across both chart understanding and table reasoning, rather than being concentrated in one dataset or model family. The gains are particularly clear on ChartMuseum across backbones, while the smaller Qwen3.5 model also benefits substantially on TableVQA-Bench; configurations that begin from stronger baselines generally show more moderate but still reliable improvements. Their consistency on the unfiltered source benchmarks supports that the improvements on OpenVisTool-Bench are not an artifact of subset selection.

G Implementation Details

Training. We fine-tune all backbones with SWIFT (Zhao et al. 2025b). Throughout training, we freeze the vision

Model	Setting	Chart		Table	
		CharXiv	ChartMuseum	TableVQA-Bench	MMTBench
Qwen3.5-4B	w/o tool	63.7	48.7	70.7	32.7
+ OpenVisTool-42K	with tool	65.5	57.8	86.7	36.5
Qwen3.5-9B	w/o tool	67.2	61.2	82.1	36.5
+ OpenVisTool-42K	with tool	68.3	66.9	88.5	39.6
Qwen3-VL-8B-Instruct	w/o tool	51.2	40.4	83.3	33.3
+ OpenVisTool-42K	with tool	54.0	46.7	84.9	36.1

Table 5: Results on the complete, unfiltered Chart and Table source benchmarks. The setting indicates whether the model is trained with tool-use trajectories; tool invocation is disabled at inference time for all models.

Hyperparameter	Setting
Epochs	3
Optimizer steps	1,971
Global batch size	64
Optimizer	Adam
Adam β_1, β_2	0.9, 0.95
Learning rate	1×10^{-5}
LR schedule	Cosine decay
Warmup ratio	0.03
Weight decay	0.1
Gradient clipping	1.0
Maximum sequence length	65,536 tokens
Maximum image tokens	8,192

Table 6: Shared training settings for all four backbones.

Hyperparameter	Setting
Temperature	0.6
Top- p	0.95
Top- k	20
Presence penalty	0.0
Repetition penalty	1.0
Maximum output length	32,768
Maximum agent turns	50
Judge model	GPT-5.5

Table 7: Evaluation and inference settings.

encoder and merger and update only the LLM parameters. The training hyperparameters are summarized in Table 6.

Evaluation. We serve the evaluated open-source models with the vLLM backend. We use the same decoding configuration for the base and fine-tuned Qwen models, as listed in Table 7. GPT-5.5 is the judge model for all judge-based tasks. GUI Grounding is scored deterministically by whether the predicted click falls inside the ground-truth bounding box.

Tool	Description
<i>General-purpose tools</i>	
read_file	Reads the contents of a file; image files are handled separately and returned as base64-encoded content.
write_file	Writes content to a specified file.
edit_file	Applies localized edits to an existing file.
list_dir	Lists the contents of a specified directory.
exec	Executes a shell command.
<i>Geometric transformations</i>	
crop	Crops a specified rectangular region from the image.
rotate	Rotates the image counter-clockwise by a specified angle, automatically expanding the canvas to fill the background.
flip	Flips the image horizontally or vertically.
<i>Annotation drawing</i>	
draw_bbox	Draws one or more rectangular bounding boxes, optionally with labels.
draw_circle	Draws one or more circles, used to mark circular targets or point locations.
draw_line	Draws one or more line segments, used to annotate axes, dividing lines, or reference lines.
<i>Feature extraction</i>	
in_range_color	Generates a mask according to an HSV or BGR color range and returns information about the matching regions.
connected_components	Extracts foreground connected components from a thresholded image, returning bounding boxes, centroids, and areas.
detect_edges	Generates an edge map using the Sobel operator, highlighting object contours, text boundaries, and structural lines.
find_contours	Extracts contours from a thresholded image, returning shape information such as area, perimeter, and bounding box.
hough_circles	Detects circular targets via the Hough transform, suitable for bubble charts, radio buttons, or circular markers.
hough_lines	Detects line segments via the probabilistic Hough transform, suitable for table lines, coordinate axes, and interface dividers.
template_match	Locates positions in the image that match a template image or a cropped template region.
<i>Image processing</i>	
adjust_brightness	Adjusts image brightness and contrast, used to lighten, darken, or enhance overall visibility.
enhance_contrast	Enhances contrast through histogram equalization, improving readability in low-contrast regions.
grayscale	Converts the image to grayscale, or further binarizes it to separate foreground from background.
<i>Visual verification</i>	
render_html	Renders an HTML file to a full-page screenshot using headless Chromium, enabling visual verification of generated HTML against a reference design.

Table 8: Complete toolset. General-purpose tools are shared across all task domains, while visual tools are grouped by functionality.

Chart VQA Tool-Use Instructions

Below are the vision tools that frequently help on chart VQA. For each tool, a concrete trigger is listed—when the situation matches, call the corresponding tool instead of guessing from the raw image.

Geometric transforms

- `crop`: when the chart contains multiple subplots, inset views, dense legends, or small tick labels, or when only a specific region (a single subplot, a legend box, an axis area) is relevant. Zooming in reduces distraction and makes labels/values legible.

Annotation / alignment aids

- `draw_line`: when reading a value off an axis—place a vertical guide at the queried x-value or a horizontal guide at the queried y-value to avoid mis-aligning bar tops / line points with the axis ticks.
- `draw_bbox`: when a specific region (a bar group, a legend entry, a highlighted area) must be tracked while cross-referencing it with the axis or legend.
- `draw_circle`: when pointing to a single data point (a scatter marker, a line peak, a pie slice) to confirm it is the one the question asks about.

Contrast / readability enhancement

- `enhance_contrast`: when grid lines, low-contrast bars/lines, small tick labels, or compressed chart details are hard to read.

Color-based lookup

- `in_range_color`: when the question depends on identifying a category, legend color, series color, or colored bar/line/area, or when estimating how much of a chart region belongs to a specific color. Prefer HSV ranges for robust selection under anti-aliasing/compression. Pass `region` to restrict matching to the plot area so legends, titles, and surrounding decorations are excluded.

Compute

- If the question involves calculation (sum, mean, ratio, ranking, percentage change, slope, etc.), use `exec` or `write_file` to create and run a Python script instead of relying on mental math.
-

Table 9: Domain-specific teacher-rollout prompt for Chart.

Table VQA Tool-Use Instructions

Below are the vision tools that frequently help on table VQA. For each tool, a concrete trigger is listed—when the situation matches, call the corresponding tool instead of guessing from the raw image.

Geometric transforms

- `crop`: when only a specific cell, row block, column block, header region, or small text is relevant, or when the image also contains surrounding captions, footnotes, or other tables. Zooming in reduces distraction and lets you read fine digits/units reliably.

Annotation / alignment aids

- `draw_bbox`: when you need to highlight and track a specific target cell or a set of candidate cells while cross-checking row label $\times$ column header $\times$ value.
- `draw_line`: when you must align a row with a column across a wide table; drawing a horizontal line across the target row or a vertical line down the target column avoids off-by-one row/column mismatches.

Contrast / readability enhancement

- `enhance_contrast`: when the table has low contrast (faded scans, light-gray zebra stripes, watermark bleed-through), small digits are hard to read, or cell backgrounds differ in subtle shades that interfere with text.

Color-based lookup

- `in_range_color`: when the question depends on cells of a specific color (highlighted rows, conditional formatting, colored status cells); the HSV range mask isolates them and returns per-component bboxes.

Compute

- If the question involves calculation (sum, mean, ratio, ranking, percentage change, etc.), use `exec` or `write_file` to create and run a Python script instead of relying on mental math.
-

Table 10: Domain-specific teacher-rollout prompt for Table.

Visual Search Tool-Use Instructions

Below are the vision tools that deliver the largest gains on visual search / grounding / counting / attribute-verification questions. When the situation matches a trigger, call the tool instead of guessing from the raw image.

- `crop`: when the target is a small object in a high-resolution image, partially occluded, distant, or surrounded by clutter. Zoom into the candidate region to verify fine attributes (color, shape, text, fine-grained category) before answering.
 - `draw_bbox`: when the question depends on locating one or more candidate objects, verifying a spatial relation (“is A to the left of B?”), or keeping track of multiple candidates during search. Drawing bboxes helps avoid missed or duplicate counting in crowded scenes.
 - `in_range_color`: when the target is defined primarily by color (“the red car”, “the blue backpack”, “all yellow flowers”); the HSV mask isolates matching pixels and returns per-component bboxes that you can then count or verify.
 - `enhance_contrast`: when the image is low-contrast (foggy, hazy, overcast, low-light indoor) and candidate objects blend into the background; CLAHE on LAB often reveals hidden targets without shifting colors.
 - `adjust_brightness`: when the image is clearly too dark (night scenes, shadows) or too bright (overexposed sky, white backgrounds with blown highlights) and the target is lost in the extreme; tune `alpha/beta` to recover details.
 - `exec`: when counting or arithmetic over detected items is required (e.g. “how many more red cars than blue cars”), collate the per-detection JSON payloads and compute the answer rather than counting by eye.
-

Table 11: Domain-specific teacher-rollout prompt for Visual Search.

GUI Grounding Tool-Use Instructions

The input is a screenshot of a GUI, and the query asks you to locate a specific UI element (e.g. “click the Submit button”, “find the search bar”, “where is the settings icon?”). Your job is to locate that element precisely and return its click position as the final answer.

Required workflow

First, use visual tools to find and verify the target. Do not guess the coordinate from the raw screenshot. Always confirm the element’s position by at least one of the tools below before committing to a final click. The coordinate you finally emit must be the **center** of the target element, not its edge or corner. Each tool call should have a clear hypothesis to confirm or reject—only call when it actually reduces ambiguity.

Finally, produce the answer as a single `computer_use` tool call with `action: “*_click”`. The `coordinate: [x, y]` must point at the center of the target element. The screen is treated as a 1000×1000 canvas, so coordinates are in the normalized `[0, 1000]` space—never return raw pixel coordinates from the original image. Emit exactly one `computer_use` call; it terminates the trajectory and is treated as your final output. Do not follow it with any other tool call or free-form text.

Pre-answer visual tools

- `crop`: zoom into the candidate region to read small labels, verify icons, or disambiguate between nearby elements. Especially important when the target is a small icon, a list item, a toolbar button, or text inside a dense layout.
 - `draw_bbox`: when multiple candidate elements exist (“the third item in the list”, “the button next to X”), or when you want to visually confirm in advance that the target you plan to click is the intended element.
 - `in_range_color`: when the target is primarily identified by color (“the red alert”, “the green confirm button”) and shape alone is ambiguous; the HSV mask returns per-component bboxes.
 - `enhance_contrast / adjust_brightness`: when the screenshot is dim, washed out, or has heavy dark-mode shadows that hide the target.
 - `detect_edges / find_contours`: when the target is defined by a thin outline (icon silhouette, table border, dividing line) that is hard to separate visually.
-

Table 12: Domain-specific teacher-rollout prompt for GUI Grounding.

HTML Code Generation Tool-Use Instructions

The task is to reproduce the webpage in the reference screenshot as faithfully as possible by emitting a single self-contained HTML document. **Use the tools below to verify and correct your draft instead of relying on a one-shot guess**—a draft that “looks right” in your head almost always diverges from the reference once rendered.

Required workflow

The trajectory must follow this shape; do not collapse steps:

1. **(Optional) Inspect the reference with vision tools first.** Call these *before* writing any HTML when they actually reduce ambiguity:
 - `crop`—when the screenshot is tall, dense, or has small text you can’t read at thumbnail level. Crop a single region (header / hero / nav / cards / footer) and look at it in isolation.
 - `in_range_color` / `sample_color`—when you would otherwise guess a hex value for a `background-color`, brand accent, button fill, or border. Sample the actual pixels and lock the palette before writing CSS.
 - `enhance_contrast` / `detect_edges`—only when the layout edges or borders are genuinely hard to see; skip otherwise.Skip this step on visually simple pages—but explain in your thinking why you can skip it. Don’t call these tools just to seem thorough.
2. **Write the first draft to a file with `write_file`.** Inline all CSS in a `<style>` block. Keep `rick.jpg` placeholders literally as-is. Use a clear filename (e.g. `index.html`).
3. **Call `render_html` on the file you just wrote.** Pass its `path`—`render_html` reads the HTML from disk, so you must `write_file` before the first `render_html` call. Whenever you can read the reference’s pixel size off the original, pass it as `viewport_width` / `viewport_height` so layout breakpoints and full-page heights match. The tool returns the rendered screenshot—visually compare it to the reference end-to-end.
4. **Name the discrepancies concretely.** After every `render_html` call, in your thinking, list the specific diffs you can see—e.g. “nav links not horizontal”, “card padding too small”, “hero image is left-aligned but should be centered”, “primary button is too saturated”. If you cannot name any concrete diff, the draft is good enough—go to step 6.
5. **Patch the HTML with `edit_file`, then re-render the same file.** Prefer `edit_file` over rewriting the whole file with `write_file`—large rewrites destroy the parts that were already correct and waste tokens. After each patch, call `render_html` on the same `path` again (the tool always reads the latest contents from disk) and re-evaluate. Iterate steps 4–5 until either the rendered screenshot is visually consistent with the reference, or a further patch is no longer closing the gap.
6. **Submit the final HTML in the assistant message.** Quote the full HTML once, then stop.

Pitfalls to avoid

- **At least one `render_html` call is required** for every trajectory—even on simple pages. The closed loop is the whole point.
 - **Patch with `edit_file`, don’t rewrite via `write_file`.** Rewriting the whole HTML between iterations destroys the parts that were already correct and wastes tokens; `write_file` is only for the initial draft.
 - **Do not reference external assets** (CDN images, Google Fonts, remote stylesheets)—the sandbox can’t reach them, the render will show broken images, and the next comparison will be misleading. Inline styles, keep `rick.jpg`-style placeholders verbatim, and rely on web-safe font stacks.
-

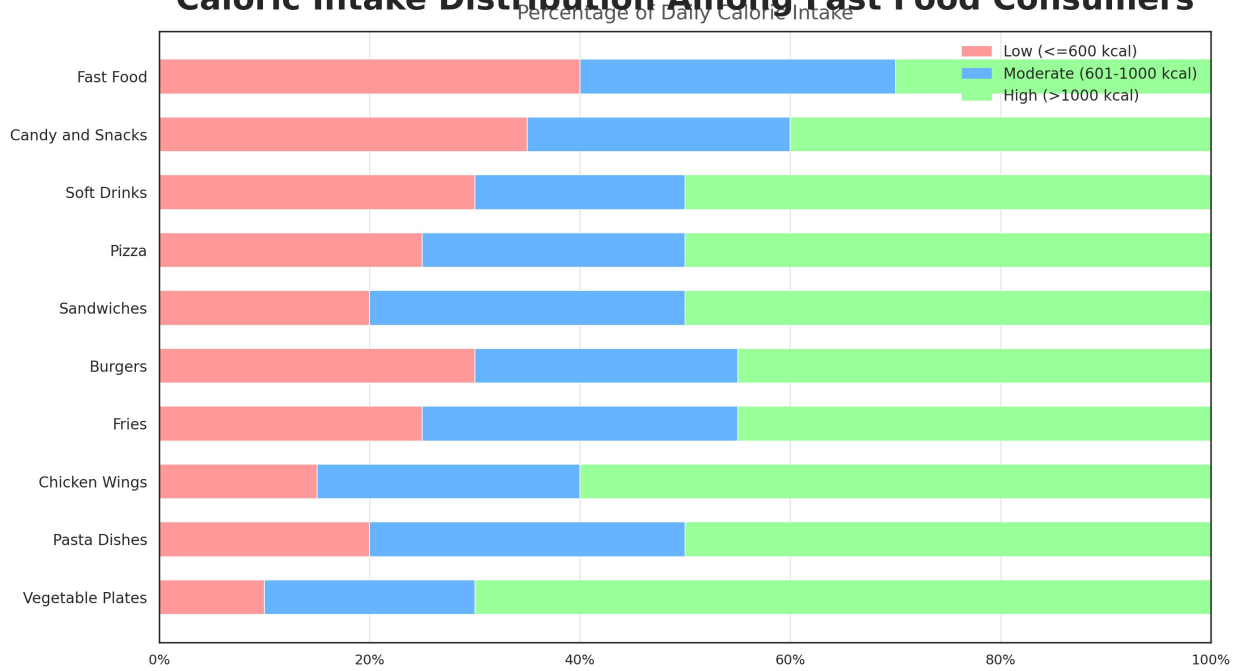
Table 13: Domain-specific teacher-rollout prompt for Web-to-HTML.

RETAINED • Chart: decompose stacked bars by color

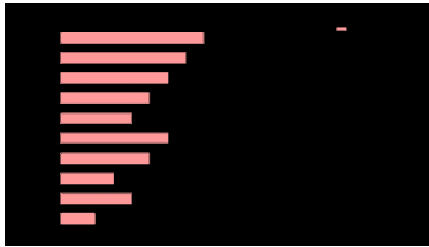
$\bar{p}_0 = 0.50 \rightarrow \bar{p}_{\text{tool}} = 0.75, g = +0.25$

Question. What is the average ratio of high-calorie consumption to combined low- and moderate-calorie consumption across all food categories?

Caloric Intake Distribution Among Fast Food Consumers



(a) Input: each category contains three adjacent segments whose widths must be measured consistently.



(b) `in_range_color`: isolate the low-calorie (pink) segment of every bar.



(c) `in_range_color`: recover the moderate-calorie (blue) widths independently.



(d) `in_range_color`: isolate the high-calorie (green) widths used in the numerator.

`in_range_color (Low)` → `in_range_color (Moderate)` → `in_range_color (High)` → `write_file` → `exec`

The three masks return aligned bounding boxes for all ten categories. For each category c , the model computes a ratio from the measured widths and then averages the ten ratios:

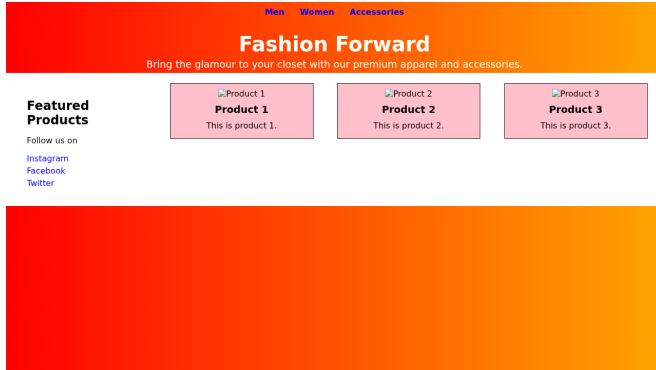
$$\frac{1}{10} \sum_{c=1}^{10} \frac{w_c^{\text{High}}}{w_c^{\text{Low}} + w_c^{\text{Moderate}}} = 1.062.$$

Figure 5: A retained Chart trajectory. Three clean color-range calls separate the low-, moderate-, and high-calorie segments of every stacked bar. Their returned bounding boxes provide the measurements used in the final script. This evidence raises the probe's avg@4 success from 0.50 to 0.75.

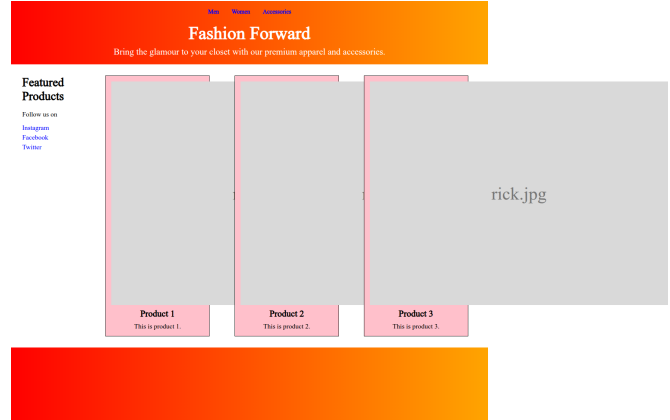
RETAINED · Web-to-HTML: render, diagnose, and revise

$$\bar{p}_0 = 0.00 \longrightarrow \bar{p}_{\text{tool}} = 1.00, g = +1.00$$

Task. Reproduce the reference webpage as a self-contained HTML/CSS file, using the prescribed placeholder for images.



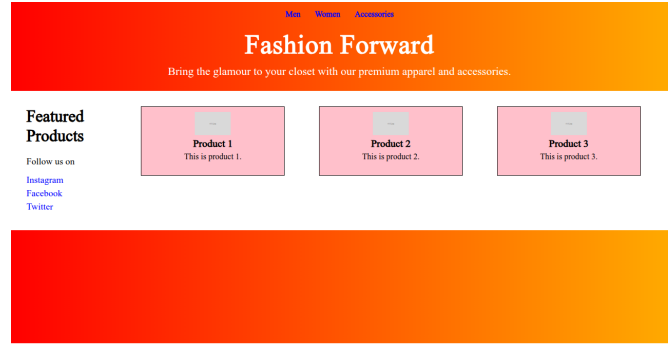
(a) Reference screenshot.



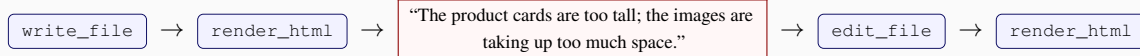
(b) First `render_html`: unconstrained placeholders overflow the product cards and push the page beyond the viewport.



(c) First revision: fixed card and image dimensions restore the intended three-card layout.



(d) Final render after spacing, typography, and footer refinements.



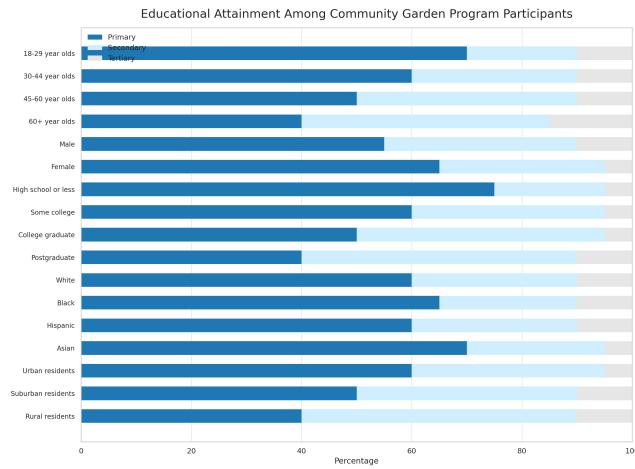
The rendered observation exposes a layout failure that is not visible in the HTML source alone. The teacher responds to that specific failure, then re-renders to verify the correction; the final page receives an HTML-VLM outcome score of 95/100.

Figure 6: A retained Web-to-HTML trajectory, condensed to the decision-changing render-revise loop. The first render reveals severe image overflow; the subsequent edit constrains the placeholders and recovers the reference’s compact horizontal layout. Later render-edit iterations refine spacing and typography.

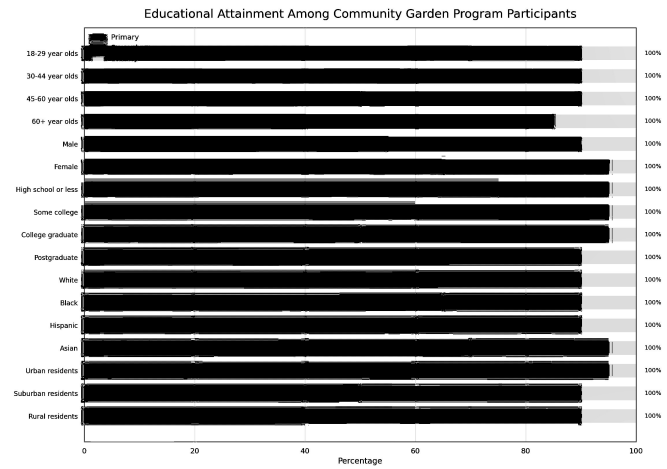
REJECTED • Outcome-valid but no causal utility

$$\bar{p}_0 = 0.50 \rightarrow \bar{p}_{\text{tool}} = 0.25, g = -0.25$$

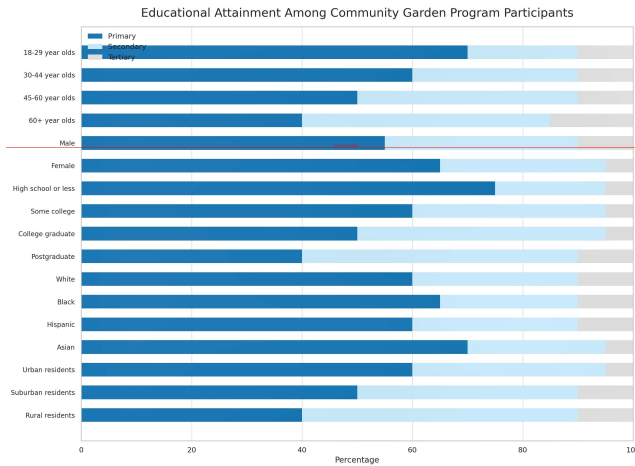
Question. In the demographic group with the highest tertiary rate, what is the combined percentage with primary or secondary education?



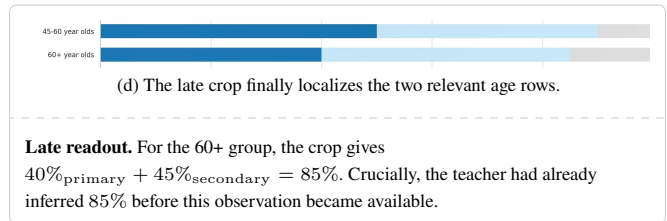
(a) Input. The teacher already estimates the target group and 85% from the raw/enhanced chart before receiving a useful localized observation.



(b) Failed `in_range_color`: an over-broad low-saturation range selects essentially the entire chart (bbox [0, 0, 1000, 1000]).



(c) Misplaced `draw_line`: the guide labeled “60+ year olds” lands on the Male row.



Why causal utility filters this trajectory

whole-chart mask → misplaced guide → late crop

Causal utility: REJECT ($g = -0.25$)

Outcome check: PASS (85%)

The answer passes correctness, but the trace is dominated by uninformative or mislocalized observations; its only useful crop arrives after the answer has effectively been inferred. Replaying the full trace lowers `avg@4` from 0.50 to 0.25, so the sample is excluded from OpenVisTool.

Figure 7: An outcome-correct trajectory removed by causal-utility filtering. The tool calls look superficially relevant, but the actual observations are uninformative or mislocalized and provide no counterfactual benefit. This case illustrates why correctness-only filtering would retain spurious process supervision: the answer is correct, yet the tool trace decreases rather than improves the probe’s success rate.